Anil Neerukonda Institute of Technology & Sciences (Autonomous)
(Affiliated to AU, Approved by AICTE & Accredited by NBA & NAAC with A+ Grade)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 08933-225083/84/87 Fax: 226395
Website: www.anits.edu.in email: principal@anits.edu.in

Department of Computer Science & Engineering (AI & ML, DS)
Mid Examination-II
CSD & CSM

Academic Year & Semester: **2025-26 Semester-I**	Program: **B.Tech**
Year: **2/4 B.Tech CSM**	Code: **23CM9201**
Course: **Java Programming Practices**	Time: **01:00 PM-03:00 P.M**
Date: **17-10-2025**	Max Marks: **30M**

NOTE: Answer any two of the following four questions, one question from each unit. Students should write the answers in serial number i.e. 1a, b, 2a, b, 3a, b, etc. and in one space in the answer book.

Q. No.	Questions	Marks
	PART-A	
1 (a)	Write a Java program to read input from the console using a BufferedReader. (**3**) convert the input string to an integer (**2**)	**5**
1 (b)	Explain about some java predefined packages (**4x1=4**) Definition of predefined packages(**1x1=1**)	**5**
1 (c)	What are the differences between Byte oriented Streams and character-oriented streams in Java(**1x5=5**)	**5**
	(OR)	
2 (a)	write a java program to Read console data by using Scanner class? (**5**)	**5**
2(b)	How to create a user defined package in java. (**5**)	**5**
2(c)	Write a program to read the text from a file using FileReader or InputStream. (**5**)	**5**
	PART-B	
3 (a)	Explain the difference between synchronized method and synchronized block. (**5**)	**5**
3 (b)	Write a program to raise an Exception a) Arithmetic Exception (**3**) b) String Index Out of Bounds Exception (**2**)	**5**
3 (c)	Write a Java program that demonstrates any 4 operations on an LinkedList of Double data. (**5**)	**5**
	(OR)	
4 (a)	Explain about Thread Priorities concept. (**5**)	**5**
4 (b)	Explain how to handle Exceptions using throws keyword. (**5**)	**5**
4 (c)	Write a Java program that demonstrates any 4 operations on an TreeSet of Character data. (**4**) TreeSet Definition(1**x1=1**)	**5**

Key

1 (a)

```
import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.io.IOException;


public class ReadIntegerExample {

    public static void main(String[] args) throws IOException {

        // Create BufferedReader object to read input from console

        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));

        // Read input as a string

        System.out.print("Enter a number: ");

        String input = reader.readLine();

        // Convert the string to an integer

        int number = Integer.parseInt(input);

        // Display the integer

        System.out.println("The number you entered is: " + number);

    }

}
```

1 (b) In Java, **predefined packages** are groups of related classes and interfaces provided by the Java Standard Library to make the development process easier. These packages contain code that can be reused in different Java programs, saving time and effort. They provide functionality for input/output (I/O), networking, utilities, and more.

1.**java.lang Package**

- This is the most fundamental package that is automatically imported into every Java program. It contains essential classes such as String, Object, Math, System, and Thread. This package does not require an explicit import statement because it is automatically imported by the compiler.

Classes and Interfaces in java.lang:

- **String**: Represents a sequence of characters.
- **Math**: Provides mathematical functions such as abs(), sqrt(), pow(), etc.

- **Object**: The root class for all classes in Java.
- **Thread**: Provides methods to control thread behavior (multithreading).
- **System**: Contains methods to interact with the runtime environment (like System.out.println()).

2. **java.util Package**

- The java.util package provides utility classes for data structures (like List, Map, Set), date and time manipulation, and random number generation. This package is widely used in Java programs.

Classes and Interfaces in java.util:

- **ArrayList**: A resizable array implementation of the List interface.
- **HashMap**: A hash table-based implementation of the Map interface.
- **Date**: Represents a specific point in time (date and time).
- **Iterator**: An interface used to iterate over collections (like List, Set).
- **Random**: Used for generating random numbers.

3. **java.io Package**

- The java.io package provides classes for system input and output, including file handling, reading from and writing to files, and data streams.

Classes and Interfaces in java.io:

- **File**: Represents a file or directory path.
- **FileReader**: A class used for reading files in character form.
- **BufferedReader**: Provides efficient reading of text from a character-based input stream.
- **InputStream**: The superclass for all classes representing an input stream of bytes.
- **OutputStream**: The superclass for all classes representing an output stream of bytes.

4. **java.net Package**

- This package provides classes and interfaces for networking, allowing Java programs to communicate over networks using sockets, URLs, and HTTP.

Classes and Interfaces in java.net:

- **URL**: Represents a Uniform Resource Locator, used to fetch resources from the web.
- **Socket**: Represents a client-side socket for communication with servers over TCP/IP.
- **HttpURLConnection**: Used to send HTTP requests and receive responses from a server.
- **InetAddress**: Provides methods to resolve hostnames to IP addresses and vice versa.
- **ServerSocket**: Used for implementing server-side socket communication.

5. **java.math Package**

- The java.math package contains classes for performing arbitrary-precision arithmetic (for large numbers), such as BigInteger and BigDecimal.

Classes and Interfaces in java.math:

- **BigInteger**: Provides support for performing arithmetic operations on arbitrarily large integers.
- **BigDecimal**: Supports high-precision decimal arithmetic, often used in financial calculations.
- **MathContext**: Specifies the context (precision, rounding mode) for BigDecimal operations.
- **RoundingMode**: Enum for specifying rounding modes used by BigDecimal.
- **ArithmeticException**: Thrown for exceptional conditions in arithmetic operations.

---

1 c. In Java, **byte-oriented streams** and **character-oriented streams** are two categories of input/output (I/O) streams that differ mainly in how they process data. These differences are essential to understand when working with file handling, network communication, and other data transmission tasks in Java.

Feature	Byte-Oriented Streams	Character-Oriented Streams
Data Type Processed	Deals with **raw bytes** of data (8 bits).	Deals with **characters** (16 bits, Unicode).
Class Hierarchy	Derived from InputStream and OutputStream classes.	Derived from Reader and Writer classes.
Used For	Used for reading and writing binary data (e.g., images, audio, etc.).	Used for reading and writing character data (e.g., text files).
Encoding Handling	Does not handle character encoding. Works with raw bytes directly.	Handles character encoding and decoding (e.g., UTF-8, UTF-16).
Examples of Streams	FileInputStream, FileOutputStream, BufferedInputStream, BufferedOutputStream	FileReader, FileWriter, BufferedReader, BufferedWriter
Performance	Faster when dealing with binary data since there is no need for encoding conversion.	May be slightly slower due to encoding/decoding characters, especially for large files.

---

2 (a)
```
import java.util.Scanner;

public class ReadConsoleData {
    public static void main(String[] args) {
        // Create a Scanner object to read input from the console
        Scanner scanner = new Scanner(System.in);

        // Reading a string input
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Reading an integer input
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Reading a double input
        System.out.print("Enter your marks: ");
        double marks = scanner.nextDouble();
```

```
        // Displaying the input values
        System.out.println("\n--- User Details ---");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Marks: " + marks);

        // Close the scanner
        scanner.close();
    }
}
```

2 (b) In Java, a **user-defined package** is a collection of related classes and interfaces that are grouped together under a specific package name. You can create your own packages to organize your classes and avoid class name conflicts, especially in large programs.

Steps to Create a User-Defined Package in Java:

1. **Define a Package**:
    - A package is defined using the package keyword at the very beginning of the Java source file.
    - The package name is followed by a semicolon. You should include the directory structure where the class is stored.
2. **Create Classes Inside the Package**:
    - After defining the package, you can create classes and interfaces inside it.
3. **Compile and Use the Package**:
    - You can compile and use the user-defined package by importing it into another class using the import statement.

step 1: Create a User-Defined Package

Create a Java file called MyClass.java that defines a package com.example.myapp.

```
// File: MyClass.java
package com.example.myapp; // Define the package name

public class MyClass {
    public void displayMessage() {
        System.out.println("Hello from MyClass in com.example.myapp package!");
    }
}
```

- In the above code, the class MyClass is part of the com.example.myapp package.

Step 2: Compile the Java File

You need to save the file in a directory that matches the package structure. In this case, it should be stored in the following directory:

com/example/myapp/MyClass.java

Then, compile the file using the javac command. From the parent directory of com, run:

javac com/example/myapp/MyClass.java

This will generate a MyClass.class file inside the com/example/myapp directory.

Step 3: Create Another Class to Use the Package

Now, let's create another Java file outside of the package to use this class. Create a file called TestPackage.java.

```
// File: TestPackage.java
import com.example.myapp.MyClass; // Import the user-defined package

public class TestPackage {
   public static void main(String[] args) {
      // Create an instance of MyClass
      MyClass myClass = new MyClass();

      // Call the method from MyClass
      myClass.displayMessage();
   }
}
```

Step 4: Compile and Run the Test Class

To compile and run the TestPackage class, follow these steps:

1. Compile the TestPackage.java class:

   javac TestPackage.java

2. Run the TestPackage class:

   java TestPackage

Output:

Hello from MyClass in com.example.myapp package!

---

2 c To read text from a file in Java, you can use either the FileReader (for character streams) or FileInputStream (for byte streams). Since you're working with text, FileReader is generally preferred as it handles characters directly, whereas FileInputStream works with raw bytes and is better suited for binary data.

Reading Text from a File using FileReader

The FileReader class is part of the java.io package and is used for reading character files. It is a convenient way to read text files.

```
import java.io.FileReader;

import java.io.BufferedReader;
```

```
import java.io.IOException;

public class FileReaderExample {

    public static void main(String[] args) {

        // Specify the path to the text file you want to read

        String filePath = "example.txt";  // Make sure the file exists

        // Try-with-resources to automatically close resources

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {

            String line;

            // Read lines from the file until EOF

            while ((line = reader.readLine()) != null) {

                System.out.println(line);  // Print each line

            }

        } catch (IOException e) {

            System.err.println("An error occurred while reading the file.");

            e.printStackTrace();

        }

    }

}
```

Reading Text from a File using FileInputStream

If you want to use FileInputStream (which works with byte data), you can manually convert the byte data into characters by using an appropriate character encoding

```
import java.io.FileInputStream;

import java.io.IOException;

public class FileInputStreamExample {

    public static void main(String[] args) {

        // Specify the path to the text file you want to read
```

```
        String filePath = "example.txt";  // Make sure the file exists

        // Try-with-resources to automatically close resources
        try (FileInputStream fis = new FileInputStream(filePath)) {
            int byteRead;
            // Read byte by byte until EOF
            while ((byteRead = fis.read()) != -1) {
                // Convert the byte into a character and print it
                System.out.print((char) byteRead);  // Print each character
            }
        } catch (IOException e) {
            System.err.println("An error occurred while reading the file.");
            e.printStackTrace();
        }
    }
}
```

---

3 (a) In Java, **synchronized methods** and **synchronized blocks** are used to ensure that only one thread can access a particular section of code at a time, preventing race conditions and ensuring thread safety.

Differences Between Synchronized Method and Synchronized Block:

Feature	Synchronized Method	Synchronized Block
Scope of Synchronization	Synchronizes the entire method, locking the method's object for the duration of the method call.	Synchronizes only a specific block of code, providing more fine-grained control over which parts of code are synchronized.
Level of Locking	Locks the object for the entire method, meaning other threads can't access any part of the method while it’s running.	Locks only the part of the code inside the synchronized block, allowing greater flexibility and finer control over synchronization.

Flexibility	Less flexible; the synchronization applies to the whole method, which might not be necessary for all parts of the method.	More flexible; synchronization can be applied to only a specific section of code that needs it, improving efficiency.
Performance Impact	Can lead to performance issues if the method is large, as the lock is held for the duration of the method.	More efficient because it minimizes the critical section, reducing the time the lock is held and improving performance.
Use Case	Useful when you need to ensure that the entire method is thread-safe.	Useful when you only need to synchronize a small part of the code inside a method, improving concurrency.

---

3 (b) a) ArithmeticException

The ArithmeticException typically occurs when there is an arithmetic error, such as dividing by zero.

```
public class RaiseArithmeticException {
   public static void main(String[] args) {
      try {
         int result = 10 / 0;  // Division by zero raises ArithmeticException
      } catch (ArithmeticException e) {
         System.out.println("Exception occurred: " + e);  // Handle the exception
      }
   }
}
```

b) StringIndexOutOfBoundsException

The StringIndexOutOfBoundsException occurs when trying to access an index of a string that is outside the valid range (either negative or greater than the string length).

```
public class RaiseStringIndexOutOfBoundsException {
   public static void main(String[] args) {
      String str = "Hello";
      try {
         char ch = str.charAt(10);  // Accessing an invalid index, beyond the length of the string
      } catch (StringIndexOutOfBoundsException e) {
         System.out.println("Exception occurred: " + e);  // Handle the exception
      }
   }
}
```

---

3 © In Java, the LinkedList class is part of the java.util package and provides methods to perform various operations on a list of elements. A LinkedList is a doubly-linked list implementation of the List.

```
import java.util.LinkedList;
```

```
public class LinkedListDemo {
   public static void main(String[] args) {

      LinkedList<Double> list = new LinkedList<>();

      // 1. Add elements to the LinkedList
      list.add(10.5);
      list.add(20.3);
      list.add(30.7);
      list.add(40.2);
      list.addFirst(5.5);  // Adds 5.5 to the beginning of the list
      list.addLast(50.8);  // Adds 50.8 to the end of the list

      System.out.println("LinkedList after adding elements: " + list);

      // 2. Remove elements from the LinkedList
      list.remove(2); // Removes the element at index 2 (30.7)
      list.removeFirst();  // Removes the first element (5.5)
      list.removeLast();   // Removes the last element (50.8)

      System.out.println("LinkedList after removing elements: " + list);

      // 3. Get elements from the LinkedList
      Double elementAtIndex1 = list.get(1);  // Get element at index 1 (20.3)
      System.out.println("Element at index 1: " + elementAtIndex1);

      // 4. Check if the list contains a specific element
      boolean contains20 = list.contains(20.3);  // Check if 20.3 is in the list
      System.out.println("List contains 20.3: " + contains20);
   }
}
```

Output: LinkedList after adding elements: [5.5, 10.5, 20.3, 30.7, 40.2, 50.8]
LinkedList after removing elements: [10.5, 20.3, 40.2]
Element at index 1: 20.3
List contains 20.3: true

---

4 (a) Thread Priorities in Java

In Java, thread priorities are used to determine the relative importance of threads during execution. The priority of a thread can influence the order in which threads are scheduled by the Java Virtual Machine (JVM). Threads with higher priorities may be executed before threads with lower priorities, but the actual behavior depends on the thread scheduling policy of the operating system.

Thread priorities are part of the Thread class in Java, and they can be set using the setPriority(int priority) method.

Thread Priority Range in Java:

- **Minimum Priority**: Thread.MIN_PRIORITY (constant value of 1)
- **Normal Priority**: Thread.NORM_PRIORITY (default value of 5)
- **Maximum Priority**: Thread.MAX_PRIORITY (constant value of 10)

These values are constants defined in the Thread class, and they represent the range of priorities that can be assigned to a thread.

How Thread Priorities Work:

1. **Thread Scheduling**:
   - The **Java Thread Scheduler** assigns CPU time to threads based on their priority and the operating system's thread scheduling mechanism.
   - Threads with **higher priorities** are more likely to be executed before threads with **lower priorities**.
   - However, thread priorities are **not guaranteed to work** consistently across all platforms. The exact behavior depends on the platform's operating system and its scheduling policy. For example, on some systems, thread priorities may be ignored, and the JVM might schedule threads without considering priority.
2. **Default Priority**:
   - By default, all threads are assigned the **normal priority** (Thread.NORM_PRIORITY), which is 5.
3. **Setting Thread Priorities**:
   - You can set the priority of a thread by using the setPriority(int priority) method, where priority is an integer between Thread.MIN_PRIORITY (1) and Thread.MAX_PRIORITY (10).

Methods for Thread Priority:

1. **setPriority(int priority)**:
   - This method is used to set the priority of a thread.
   - The parameter should be an integer between Thread.MIN_PRIORITY (1) and Thread.MAX_PRIORITY (10).

   ```
   Thread t = new Thread();
   t.setPriority(Thread.MAX_PRIORITY);  // Set the thread's priority to maximum (10)
   ```

2. **getPriority()**:
   - This method retrieves the current priority of a thread.

   ```
   int priority = t.getPriority();  // Get the current priority of the thread
   ```

---

4 (b) In Java, exceptions can be handled using two primary approaches: using try-catch blocks or using the throws keyword. The throws keyword is used to declare that a method can potentially throw one or more exceptions, and it's a way of **passing the responsibility** for handling the

exception to the calling method. This is useful for cases where a method cannot or should not handle the exception itself, but wants to allow the caller of the method to handle it.

The throws keyword in Java is used in a method declaration to indicate that the method **might throw certain exceptions** during its execution. This is a way for the method to communicate to the calling code that it may raise specific exceptions, and the caller must either handle or propagate those exceptions further up the call stack.

Syntax of throws

The basic syntax of throws is as follows:

```
returnType methodName() throws ExceptionType1, ExceptionType2 {
   // method body
}
```

- ExceptionType1, ExceptionType2: These are the exceptions that the method might throw.
- The calling method is responsible for catching and handling or further propagating the exception.

---

4 © In Java, a TreeSet is a part of the java.util package and implements the Set interface. It is a **NavigableSet** that uses a **Red-Black tree** for storage. A TreeSet stores elements in **sorted order**, and it does not allow duplicate elements. Since it implements the Set interface, it doesn't allow repeated elements, and it automatically sorts elements in their natural order (or according to a comparator if one is provided).

```
import java.util.TreeSet;

public class TreeSetOperations {

   public static void main(String[] args) {

      TreeSet<Character> charSet = new TreeSet<>();

      // 1. Add elements to the TreeSet

      charSet.add('A');

      charSet.add('C');

      charSet.add('B');

      charSet.add('D');

      charSet.add('E');

      // Display the TreeSet (automatically sorted)

      System.out.println("TreeSet after adding elements: " + charSet);

      // 2. Remove an element from the TreeSet
```

```
        charSet.remove('C'); // Removing element 'C'

    // Display the TreeSet after removal

        System.out.println("TreeSet after removing 'C': " + charSet);

        // 3. Check if an element exists in the TreeSet

        boolean containsB = charSet.contains('B'); // Checking if 'B' is in the set

        System.out.println("TreeSet contains 'B': " + containsB);

        // 4. Display the sorted TreeSet (this operation is implicit since the TreeSet maintains
order)

        System.out.println("Sorted TreeSet: " + charSet);

    }

}
```

Output: TreeSet after adding elements: [A, B, C, D, E]

TreeSet after removing 'C': [A, B, D, E]

TreeSet contains 'B': true

Sorted TreeSet: [A, B, D, E]